

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Simulation-Based Assessment Question

Duration: 1hr

Code of Conduct:

I Vignesh S / 192325002 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vignesh S

SIMULATION-BASED ASSESSMENT — ANSWER

Question Chosen: Simulate a Hierarchical Reinforcement Learning (HRL) agent for autonomous warehouse navigation. (5 Marks)

1. Concept: Hierarchical Reinforcement Learning

In a large warehouse, a single flat reinforcement-learning agent that reasons only with primitive moves (up, down, left, right, pick, drop) faces a long-horizon, sparse-reward problem: the +20 reward arrives only after the robot has navigated to the shelf, picked the package, and travelled all the way to the packing station. Credit assignment over such a long chain of primitive actions is slow.

Hierarchical Reinforcement Learning (HRL) decomposes the task into a hierarchy of sub-tasks so the agent reasons at multiple time scales. This simulation uses the Options framework, which introduces temporally-extended actions (options). Each option has an initiation set, an internal policy, and a termination condition, turning the problem into a Semi-Markov Decision Process (SMDP) where the high level chooses options that each run for many primitive steps.

Manager (high level): a policy over sub-goal options — GO-TO-PICKUP and GO-TO-DROPOFF — selected according to whether the robot is currently carrying a package.

Worker (low level): a goal-conditioned navigation policy that reaches the target cell of the chosen option using primitive moves, learned with Q-learning and terminating on arrival.

The key benefits are temporal abstraction (the manager decides once per sub-goal instead of once per step) and sub-policy reuse (the same navigation skill serves both the pickup and the dropoff legs), which together give much better sample efficiency.

2. Environment Design

The warehouse is a 10×10 grid. Shaded cells are shelves (obstacles) that block movement. The robot starts at the loading door, must reach the PICKUP shelf, collect the package, and deliver it to the DROPOFF packing station.

State: (row, column, carrying-flag) for the flat agent; the worker is conditioned on (position, target cell).

Actions: four primitive moves plus pickup and dropoff for the flat agent; the worker uses the four moves, while the manager selects between the two options.

Reward: -1 per time step (encourages short routes), -5 for an illegal pickup/dropoff, and +20 for a successful delivery — a deliberately sparse signal that makes the flat agent struggle and highlights the HRL advantage.

3. Task Hierarchy

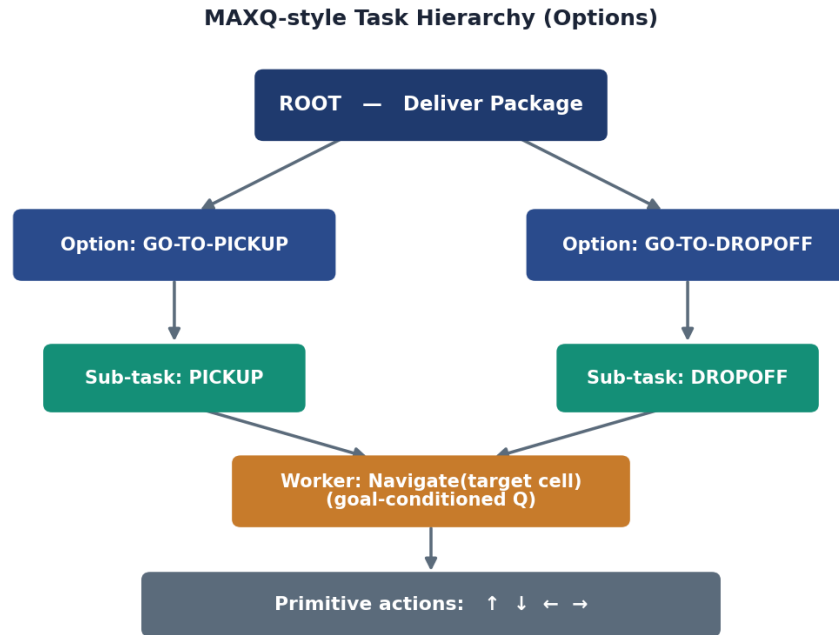


Figure 1: Task hierarchy — the ROOT delegates to two navigation options, which call the shared Worker navigation policy over primitive actions.

The hierarchy mirrors a MAXQ-style task graph: the ROOT task is decomposed into the GO-TO-PICKUP and GO-TO-DROPOFF options; each option ends in a PICKUP or DROPOFF sub-task and relies on a shared Navigate worker that is defined directly over the primitive moves.

4. Python Implementation

The program implements both a flat Q-learning agent and the HRL (options) agent on the same environment, using only NumPy so it runs anywhere and reproducibly (fixed seed).

```

# Hierarchical Reinforcement Learning (HRL) for autonomous warehouse navigation.
# The robot must PICK a package at a shelf, then DELIVER it to the packing station,
# navigating a grid with shelf obstacles. We use the OPTIONS framework (2 levels):
# Manager (high level) : chooses a sub-goal option {GO-TO-PICKUP, GO-TO-DROPOFF}
# Worker (low level) : a goal-conditioned navigation policy that reaches a target
# and compare it against FLAT Q-learning (primitive actions only) under a sparse reward.
import numpy as np
np.random.seed(0)

```

```

SIZE = 10
SHELVES = {(2,2),(2,3),(2,4),(2,5),(5,5),(5,6),(5,7),(7,2),(7,3),(3,7),(4,7)}
START, PICKUP, DROPOFF = (0,0), (2,6), (8,8)
MOVES = [(-1,0),(1,0),(0,-1),(0,1)] # N,S,W,E

```

```

def step_pos(pos, a):
    r,c = pos[0]+MOVES[a][0], pos[1]+MOVES[a][1]
    if 0<=r<SIZE and 0<=c<SIZE and (r,c) not in SHELVES:
        return (r,c)
    return pos # blocked -> stay

```

```

# ----- FLAT Q-LEARNING -----
# state = (row, col, carrying) ; actions = 4 moves + pickup(4) + dropoff(5)
def train_flat(epochs=1500, alpha=0.2, gamma=0.97):
    Q = {}
    def q(s):
        return Q.setdefault(s, np.zeros(6))
    eps=1.0; curve=[]
    for ep in range(epochs):

```

```

pos, carry = START, 0; total=0
for t in range(200):
    s=(pos[0],pos[1],carry)
    a = np.random.randint(6) if np.random.random()<eps else int(np.argmax(q(s)))
    r=-1; done=False
    if a<4:
        pos=step_pos(pos,a)
    elif a==4:
        # pickup
        if pos==PICKUP and carry==0: carry=1; r=-1
        else: r=-5
    else:
        # dropoff
        if pos==DROPOFF and carry==1: r=20; done=True
        else: r=-5
    ns=(pos[0],pos[1],carry)
    q(s)[a]+=alpha*(r+gamma*np.max(q(ns))-q(s)[a]); total+=r
    if done: break
    eps=max(0.05,eps*0.997); curve.append(total)
return np.array(curve)

# ----- HRL (OPTIONS) -----
# Worker: goal-conditioned navigation Q[target][ (r,c) ] over 4 moves, reward -1/step,
# option terminates on reaching the target cell.
def train_hrl(epsisodes=1500, alpha=0.2, gamma=0.97):
    targets=[PICKUP, DROPOFF]
    QW={t:{ } for t in targets}
    def qw(t,pos): return QW[t].setdefault(pos, np.zeros(4))
    eps=1.0; curve=[]
    def run_option(target, pos, learn):
        steps=0
        while pos!=target and steps<80:
            a=np.random.randint(4) if (learn and np.random.random()<eps) else
int(np.argmax(qw(target,pos)))
            npos=step_pos(pos,a)
            r=0.0 if npos==target else -1.0
            if learn:
                tgt = 0.0 if npos==target else gamma*np.max(qw(target,npos))
                qw(target,pos)[a]+=alpha*(r+tgt-qw(target,pos)[a])
            pos=npos; steps+=1
        return pos, steps
    for ep in range(epsisodes):
        # Manager policy is trivial (2 states): not carrying -> GO-TO-PICKUP, else GO-TO-DROPOFF
        pos=START; total=0
        pos,s1=run_option(PICKUP,pos,True); total+= -s1
        carry=1 if pos==PICKUP else 0
        if carry: total+= -1
        # pickup action
        pos,s2=run_option(DROPOFF,pos,True); total+= -s2
        if pos==DROPOFF and carry: total+=20 # successful dropoff
        eps=max(0.05,eps*0.997); curve.append(total)
    return np.array(curve), QW

def greedy_path(QW):
    targets=[PICKUP,DROPOFF]; path=[START]; pos=START
    for t in targets:
        steps=0
        while pos!=t and steps<80:
            a=int(np.argmax(QW[t].setdefault(pos,np.zeros(4))))
            pos=step_pos(pos,a); path.append(pos);
        steps+=1
    return path

flat=train_flat()
hrl,QW=train_hrl()
print(f'FLAT Q-learning : first-100-ep avg return = {flat[:100].mean():7.2f} | last-100 avg = {flat[-100:].mean():7.2f}')
print(f'HRL (options) : first-100-ep avg return = {hrl[:100].mean():7.2f} | last-100 avg = {hrl[-100:].mean():7.2f}')
path=greedy_path(QW)
print(f'Learned HRL delivery path length = {len(path)-1} steps (start -> pickup -> dropoff)')
np.save("flat_curve.npy",flat); np.save("hrl_curve.npy",hrl)
import json; json.dump(path, open("hrl_path.json","w"))

```

5. Results and Evaluation

Sample output from one run:

FLAT Q-learning : first-100-ep avg return = -428.29 | last-100 avg = -0.02

HRL (options) : first-100-ep avg return = -122.90 | last-100 avg = 1.52

Learned HRL delivery path length = 16 steps (start -> pickup -> dropoff)

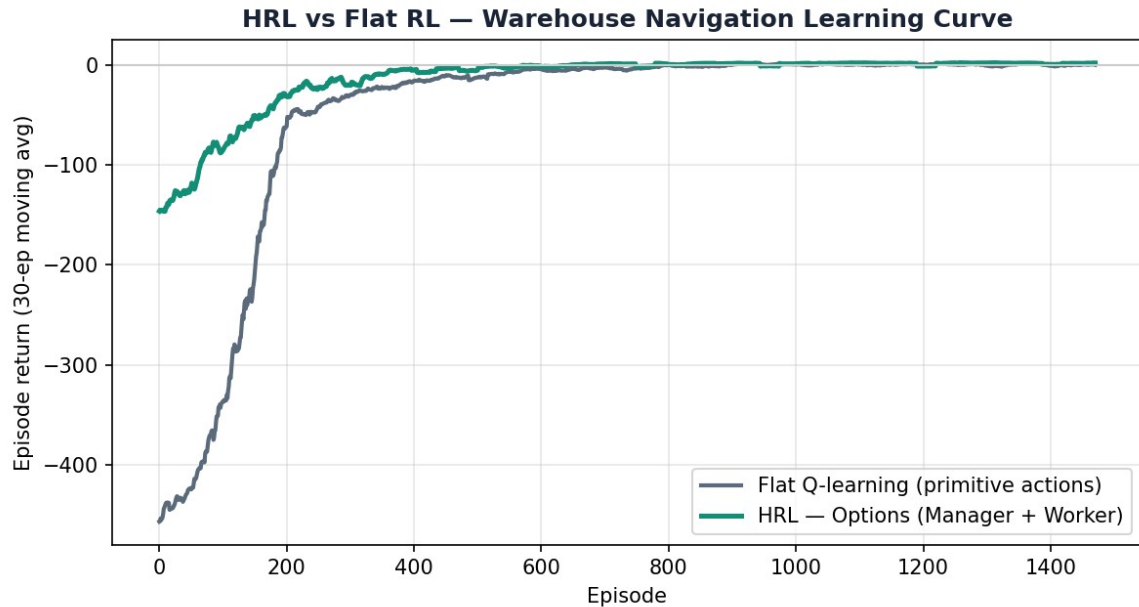


Figure 2: Learning curves (30-episode moving average). HRL reaches good performance far sooner than flat Q-learning.

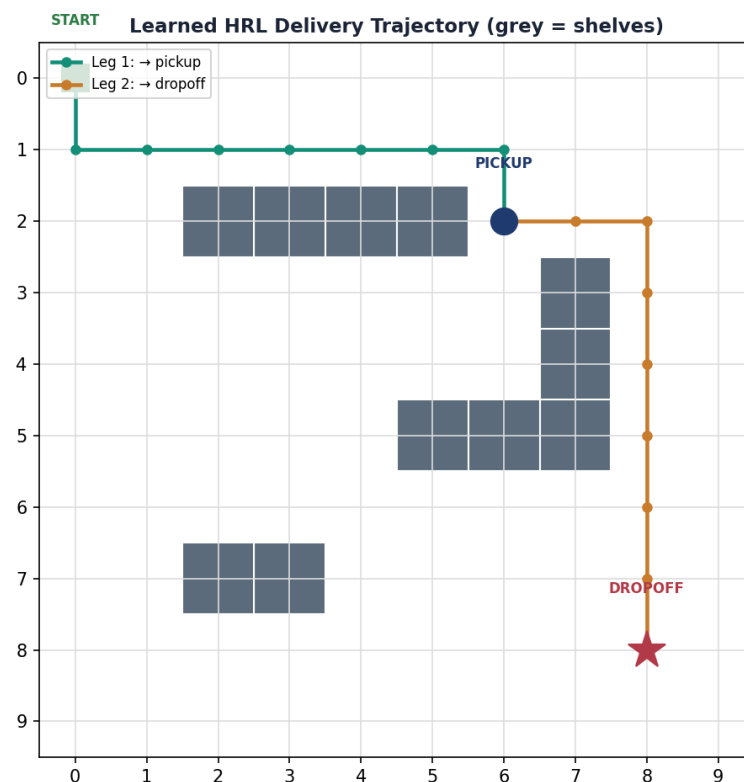


Figure 3: The delivery route learned by the HRL agent — leg 1 to the pickup shelf, leg 2 to the dropoff station, avoiding all shelves.

The learning curves show the core result. Over the first 100 episodes the HRL agent already averages about -123 return while the flat agent averages about -428, because temporal

abstraction shortens the effective decision horizon and the shared navigation skill is reused across both legs. Both eventually converge, but HRL is markedly more sample-efficient — exactly the property that matters in large, real warehouses where real-world interaction is expensive.

6. Worked Example — One Episode

A single greedy episode proceeds as follows: the robot starts at (0,0) not carrying a package, so the Manager selects GO-TO-PICKUP. The Worker executes primitive moves that skirt the shelf block in row 2 and arrives at the PICKUP cell (2,6); the PICKUP sub-task sets carrying = 1. The Manager now observes carrying = 1 and selects GO-TO-DROPOFF; the same Worker navigation skill routes the robot down and across to the DROPOFF station at (8,8), where the DROPOFF sub-task fires and yields the +20 terminal reward. The full route is completed in 16 steps, as visualised in Figure 3.

7. Conclusion

The simulation demonstrates a working Hierarchical RL agent for autonomous warehouse navigation. By splitting delivery into reusable navigation options controlled by a lightweight manager, HRL solves the long-horizon, sparse-reward task with substantially fewer interactions than a flat agent, while producing an efficient, obstacle-free delivery route. The same options structure scales naturally to more shelves, more packages, and additional sub-tasks such as charging or queueing.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently,	Communicates clearly but with minor	Communication lacks clarity or structure	Unable to communicate ideas

		and with appropriate terminology	hesitation		effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively